

CDQN: Context infused Sequential Object Detection with Deep Reinforcement Learning in Aerial Images

Rohit Gandikota

¹Northeastern University
Boston, MA

Abstract

This manuscript presents a method for detecting objects in satellite images using deep reinforcement learning. Object detection in aerial/satellite images presents unique challenges compared to real-world images due to the relatively small size of objects compared to the large size image. This makes it difficult for one-shot object detectors like YoLo to make accurate detections, and region-based proposal methods like R-CNNs are computationally complex and less accurate in the satellite image domain. Our method uses a sequential hierarchical agent that starts with the entire image as a bounding box and focuses on pre-defined fixed regions within the image, zooming the bounding box to these regions. The image is then cropped to the new bounding box, and the process is repeated until the agent terminates into a final bounded box. We propose working with multi-resolution images to address the huge computational requirement associated with satellite images. We start from a lower-resolution image, and the agent can increase the resolution based on the current state information. The infused contextual features allow the agent to make more accurate predictions at lower resolutions. Moreover, our work compares various DQN settings and argues that adding a history vector of actions to the state is sub-optimal, as suggested by prior works. Experiments also suggest that having overlapping states tend to better performance. Source code and models are available at <https://github.com/RohitGandikota/detection-2022-CDQN>

Introduction

Traditionally in vision, images have been analysed by a sliding window approach. Deep learning usually occurs at different kernel sizes (essentially at different scales). However, this approach considers every sub-part independent of each other, drawing no relation between them. We propose a hierarchical way to exploit these relations between sub-images such that the overall search is not brute-force. Specifically, we propose a top-down search approach, where the detection starts from a global image and then is sequentially focused on local regions guided by context.

This work is inspired by how humans detect objects in an image. We always perform in a sequential way, where we tend to look at the regions with salient features. We then perform a finer detailed search for specific objects. For instance,

Copyright © 2022, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

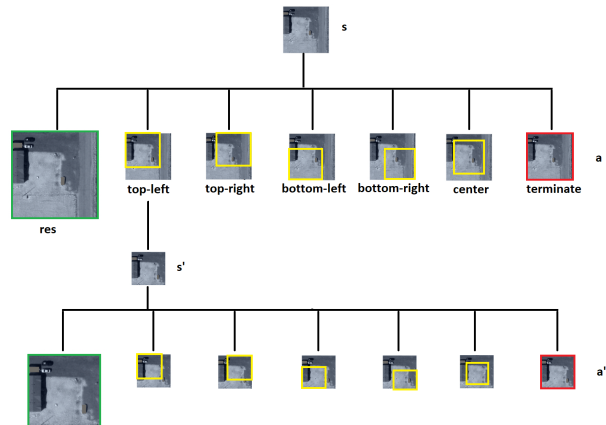


Figure 1: An illustration showing the current state on the top row and the possible actions on the second row. In this example, the agent chooses *top-left* action, leading to the next state (shown in the third row). It is a sequential process from there on.

if a satellite image is given and a human is asked to search for a boat, naturally, we look at regions with water first, zoom the image and, start looking for ablations in water, then finally detect ships/boats. This is the mechanism that we tried to mimic using contextual features. Images might not have all the information required to find finer objects at lower resolution. We, therefore, infuse context such that the model knows to zoom into "water" first and then ask for higher-resolution water patches.

Our work is based on a deep Q network agent capable of detecting objects in satellite images. This work is inspired by (Bueno et al. 2017), where they use sequential object detection in natural images from PASCAL-VOC dataset (Everingham et al. 2010). Our work extends this idea and incorporates contextual features that enable the agent to learn in multi-dimensional still images. The infused context into the Q network helped the agent focus on local regions with a high likelihood of containing a particular object. Once the agent zooms into such regions, it requires a higher resolution but smaller images (as we zoom into local regions). This enables the model to perform faster and better in a dynamic

environment like satellite images.

To elaborate, the agent first analyses the entire image based on its contextual features. It then focuses on a particular region from a predefined set of regions. We choose the four quadrants in the image and the central region for simplicity. Marriam et al. (Bueno et al. 2017) suggest that overlapping quadrants tend to focus much better to avoid missing objects at the intersection of quadrants. To reduce complexity, we start with a lower-resolution image, the agent, however, can request a higher-resolution image incurring a slight negative penalty. Finally, the agent can terminate based on the current state. It is important to note that the state is the current image (a zoomed sub-region of the original image); once the agent terminates, the current image bounds are set as a bounding box. In this case, the need for reinforcement learning can be overlooked; however, we would like to emphasise the different ways this result can be achieved. An RL agent can make a smarter decision based on the expected final return; this way, the method will be faster, more efficient and lighter.

The first contribution of this work is to introduce contextual satellite features that help in faster learning. The second contribution is the introduction of resolution scaling in action space allowing the agent to work in lower resolution with the help of context. Finally, we introduce the power of double DQN to avoid maximization bias in learning. In this manuscript, we briefly discuss the previous work and then discuss the methodology in detail. We later presented our results, followed by some analysis of the computation and efficiency. We conclude by discussing various experiment settings and their advantage using ablation studies.

Related Work

Reinforcement learning is a powerful tool used in various applications, including achieving superhuman performance in playing Atari 2600 (Mnih et al. 2015) video games and winning the Go competition against a professional player (Silver et al. 2016). In the field of traditional computer vision tasks, reinforcement learning has been applied to image classification (Ba, Mnih, and Kavukcuoglu 2014; Mnih et al. 2014), captioning (Ba et al. 2015), activity recognition (Yeung et al. 2016), and object detection (Caicedo and Lazebnik 2015; Bueno et al. 2017).

Traditional methods like selective search (Van de Sande et al. 2011) is based on sliding window classification. These classification scores allow the network to choose a bounding box. These methods are supervised methods and can be sequential in the process by turning the sliding windows to independent edgebox predictions (Zitnick and Dollár 2014). These methods are computationally exhaustive owing to their large region proposals and classifications. Recent methods like Faster-RCNN have harnessed the power of global features but heavily rely on reference boxes called anchors. These methods regress over the initial anchors to predict better bounding boxes. We take the key idea of refining bounding boxes, but instead of regressing multiple boxes, we refine a single bounding box with a reinforcement learning agent.

Yoo et al. (Yoo et al. 2015) propose a classification-based sequential detection method that refines a single box us-

ing classification scores. These scores are the basis of weak directions pointing to the target. Caicedo et al. (Yoo et al. 2015) propose an active localization using deep reinforcement learning. Their method manipulates the bounding box with each action changing the aspect ratio. This makes the problem a continuous action MDP, but this work discretises the space to work with DQN. We find this sub-optimal and propose using pre-determined static hierarchical quadrants that force the agent to search top-down by zooming into the regions.

Miriam et al. (Bueno et al. 2017) introduced hierarchical representation to top-down guide the agent through the image. They explored how the design of the hierarchy affects the detection performance and the number of visited regions. They propose the usage of history vectors for smoother trajectories; however, we find them to be sub-optimal (more in the analysis section). They also use direct vanilla DQN with no target network, making the learning unstable. We propose the usage of double DQN with context. We also propose the usage of resolution scaling to address the huge computation requirement of satellite images.

Method

In this section, we formally define the Markov Decision Process (MDP) and then discuss the Q learning setup and the model architecture. We finally discuss the training procedure that was designed for this work.

MDP Formulation

To clearly understand the crux of this work, we first define the Markov decision formulation.

State The state is designed to be the contextual feature of the image. We try two methods, one with appended action history vector and the other without. This is done to study the effect of history vectors as proposed by (Bueno et al. 2017). The memory vector consists of one-hot encodings of the last 4 actions taken by the agent. This vector is proposed by (Bueno et al. 2017) to stabilize the search trajectories. There are 7 actions in total, and we considered 4 actions for history; this makes the history vector size 28. This is clearly shown in Figure 2.

Actions We define 3 classes of actions: resolution upscale (*res*), movement actions, and terminal action. The resolution upscaling *res*, refers to the agent asking for a higher resolution patch of the current image. There are five different movement actions *top-left*, *top-right*, *bottom-left*, *bottom-right*, and *center*. These refer to the agent focusing on sub-regions of the current image. These actions will lead to a next state with a smaller image. These five actions are shown in Figure 1 in yellow regions. Finally, terminal action *terminate* will lead to the end state of the MDP. This current state will be the final predicted bounding box by the agent if it takes *terminate* action. This is shown in the red box in Figure 1.

Reward For the reward, we consider 3 different reward scenarios. First, reward that corresponds to movement actions. When a movement action is taken by an agent, we present a positive reward if the agent focuses on better re-



Figure 2: A representation of how the state is calculated based on contextual network. As can be seen, the image is first passed through the contextual network to obtain the contextual features (in blue). Finally, a vector of action history (in red) is added to these features to form the state vector.

gions and a negative reward if the agent focuses on poor regions. We measure prediction quality using the IOU (Intersection Over Union) metric. We observed that simply using the difference of the next state bounding box to the current state bounding box is a weak reward signal for the network to learn. Therefore, we use the *sign* function to measure reward, essentially giving a +1 reward if the difference in positive, else -1. This can be seen in Equation 1, where s and s' are the current and next state bounding boxes, respectively. gt refers to the original ground truth bounding box.

$$r_{mov}(s, s', gt) = \text{sign}(\text{IOU}(gt, s') - \text{IOU}(gt, s)) \quad (1)$$

Similarly, we define reward associated with resolution upscaling. We give a small negative reward to discourage frequent upscaling of resolution as it increases the computational burden. For a fair reward signal, we give a relatively negative reward based on the patch size. We define this reward in Equation 2, where $h_{s'}$, $w_{s'}$ refers to the height and width of the higher resolution image, and the constant α is a normalising factor where we chose it as the original image size. This makes the $-1 < \text{reward}_{res} < 0$.

$$r_{res}(s, s') = -\frac{h_{s'} \times w_{s'}}{\alpha} \quad (2)$$

Finally, the termination reward is based on the quality of the final bounded box. This essentially uses the IOU of the final bounding box and thresholds the value. If the final IOU is greater than a threshold, we give a positive reward $+\tau$; else, we give a negative reward $-\tau$. This is defined in Equation 3. We choose the τ as 3 and θ as 0.5 as it is better for learning.

$$r_{term}(s, gt) = \begin{cases} +\tau & \text{if } \text{IOU}(s, gt) \geq \theta \\ -\tau & \text{otherwise} \end{cases} \quad (3)$$

Transitions As it is clear, we choose deterministic transitions for predictable and smoother transitions. Since we have control over the environment, we choose a deterministic environment. However, one can easily extend this work to add stochastic nature to the transitions. Therefore, as shown in Figure 1, each action shown in the yellow box will deterministically lead to the next state that zooms into the corresponding sub-region as shown in the tree.

Discount Factor It is common to define the discount factor in MDP formulation. We choose a γ of 0.9 to consider the future rewards that could incur at the current action. Choosing γ high led to a good balance between immediate and future rewards.

Double Q Learning

The agent might look like it learns to take a particular action at a state based on the reward. However, underneath the hood, we train the agent to calculate $Q(s, a)$ values called state-action values. These values consider not only immediate rewards but also consider future returns. These Q values can be calculated using a standard tabular way or function approximation. The tabular method would require keeping track of all the state action pair values. In our case, the state-action space is too large to keep track of ($131026 \times 8 \times 255$). Therefore, we use the function approximation of Q values using deep neural networks. The update of the sample-based, 1-step, tabular Q values is shown in Equation 4. γ represents the discount factor. This factor considers priority given to immediate rewards compared to future rewards.

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)) \quad (4)$$

Since we use a function approximator (deep convolutional network) to predict the Q value, we update the parameter λ as shown in Equation 5

$$\lambda \leftarrow \lambda + \alpha(r + \gamma \max_{a'} Q_{\lambda}(s', a') - Q_{\lambda}(s, a)) \nabla_{\lambda} Q_{\lambda}(s, a) \quad (5)$$

This method, used by (Bueno et al. 2017), was found to be sub-optimal as the learning was unstable and seemed to be steered by maximization bias. One issue with deep Q networks and Q learning algorithms is the tendency to over-estimate Q values. This means that as our estimates of Q values approach their true values, they may shift because we use the same network to generate both values. This could be seen in the training of our agent, where the agent always asks for a higher-resolution image. Therefore, we propose using double-DQN, where we maintain a copy of the original network (the target network with network parameters λ') and update it less frequently than the main network. This way, we can avoid any bias from the main network as the target network pulls the bias back. This mechanism is represented in Equation 6

$$\begin{aligned} a' &= \text{argmax}_{a'} Q_{\lambda}(s', a') \\ \lambda &\leftarrow \lambda + \alpha(r + \gamma Q_{\lambda'}(s', a') - Q_{\lambda}(s, a)) \nabla_{\lambda} Q_{\lambda}(s, a) \\ \lambda' &\leftarrow \lambda (\text{every 4000 steps}) \end{aligned} \quad (6)$$

Model Architecture

Our work considers a deep neural network (fully connected network) to approximate Q values. To achieve this, we first

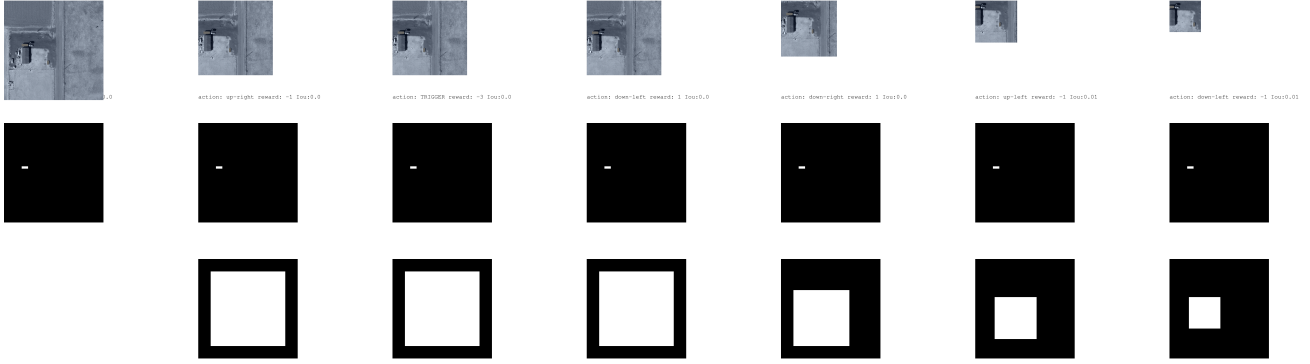


Figure 3: A sample run of the CDQN agent on the VEDAI dataset. The columns represent the timesteps in the episode. The top row represents the states at each time step. The second row represents the ground truth bounding box at the current state. The last row represents the predicted bounded box relative to the image.

pass the image through a contextual network as shown in Figure 4. We use the pre-trained feature extractor from RTC-GAN(Gandikota et al. 2020) for contextual networks. This was our previous work where the model was used for LULC classification of the satellite images. Using this network enabled the model to get contextual information like the location of roads, water, and vegetation. This way, at lower resolutions, the agent could still have the necessary information to zoom into a region. This model is a residual neural network with 7 contracting convolution layers (64, 128, 256, 512, 512, 512 and 512 filters).

For the Q network, we use a simple 2 linear layer layers of dimension 1024. We choose a lighter network compared to the previous work (Bueno et al. 2017) as they use the image directly in the Q network compared to simplified contextual features in our case. We used ReLU activation with alpha 0.2 and randomly initialized the weights sampling from a normal distribution of 0 mean and 0.02 standard deviation.

Training

We discuss the training details that were designed for this specific model.

Multi-object Image We observed during training that specifying a particular object inside an image led to poor training. Instead, we let the agent focus on regions it deems to be "high likelihood" based on its Q values. This was designed by calculating IOU with a bounding box with a high intersection with the current state predicted by the agent. We also mask out the detected objects in the image to push the agent to focus on other objects inside the particular image.

Hyper Parameters We choose to compare the effects of action-history vectors in state representation. We chose the 4 actions to append as this was good, giving initial results. We let the agent run to 14 e per episode; this avoided the zoomed images from going to 0 sizes. We updated the target vector every 100 steps. We train the agent for 50 train epochs, each containing 1250 sample epochs (62000 episodes). We choose the Adam optimizer with a 1e-2 learning rate and random uniform initialization.

Replay Buffer Since we use a function approximation, sequentially training the network with consecutive samples leads to inefficiency in training as the samples would be highly correlated. We, therefore, use a buffer to store the (s, a, r, s') pair and call it the replay buffer. This is done by simulating the agent; we start from an image state s , take action a based on Q values ($\arg\max_a Q(s, a) + noise$), observe the reward r and next state s' and store the tuple in a buffer. It is to be noted that we do not store the image but the contextual features as defined above. We choose a size of 1000 buffers; once the buffer is full, we start training the network by randomly sampling pairs from the buffer. We replace the pair with the least error with new samples, essentially making it a pseudo-prioritized buffer.

Exploration At the beginning of the training, the network would not have any clue about the task or the environment. Therefore it is important to explore the actions at the beginning. Therefore we add an exploration constant ϵ which anneals over time as the agent learns more and more about the task. ϵ refers to the probability at which the agent chooses random action compared to optimal action. This is the *noise* term that was defined above.

Experiments

We set our experiments on 2 different datasets, PASCAL-VOC 2012(Everingham et al. 2010) and VEDAI (Razakarivony and Jurie 2016). PASCAL dataset, with 5000 samples, is used to compare the current methods over our method to observe the advantages of history vector, double Q learning and training time. We use the VEDAI dataset, with 1200 samples, to compare the effectiveness of our method on aerial images to detect vehicles.

Qualitative Results

We present some visual results in Figure 3. These results are from an agent with a history vector and 3/4th overlapping quadrants. We observed that most images had zoomed semantically well, but there were instances where the images were overshoot into regions without any object. This usually

Contextual-DQN

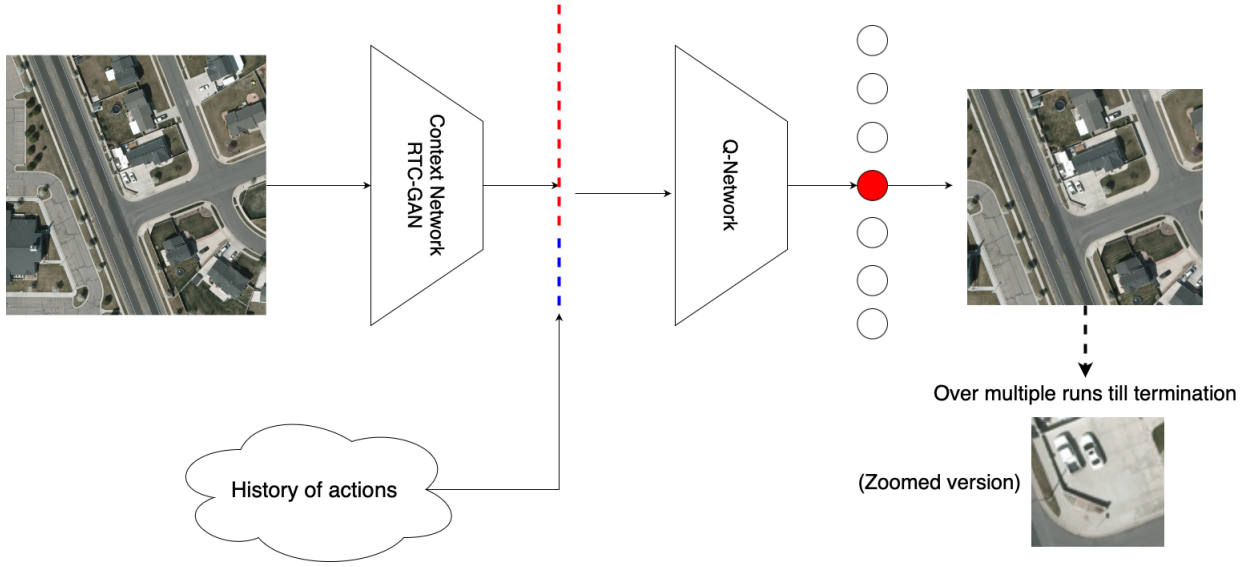


Figure 4: The proposed architecture for Contextual-DQN. The model consists of a contextual network (for context-based guidance) and a Q network (to approximate the Q values). The highlighted node in red colour represents the maximum Q value corresponding to action "center" zoom. Therefore the next state is a zoomed version of the central region.

occurs when the agent does not upscale the resolution, as shown in Figure 5. The columns represent the time step in each episode. The top row shows the image state, the middle row shows the ground truth image, and the bottom row shows the predicted bounding box relative to the original image.

Computation Efficiency

Our method is inspired by the work (Bueno et al. 2017) with major differences in the state space representation, usage of double Q networks, and prioritized buffers. The main improvement is the training efficiency. We use a contextual network to reduce the feature space to lower dimensional information. This not only helped stabilise the training but also helped to make the Q networks lighter. We compare training and testing efficiency in Table 1 over the PASCAL dataset. It is to be noted that for the PASCAL dataset, we use pre-trained ResNet-50 features on Imagenet as the context would be closely related. It can be seen that the training times are improved by 50% while the IOU has improved significantly. We also compare our methods with RCNN to show the huge differences in the computation time. It can be seen that RCNN are efficient than our method but takes more than 10 times the test time, which calls for huge concern when dealing with satellite images.

Performance over Aerial Images

Now we address the main problem statement of detecting objects in satellite images. As it can be seen in Table 2,

Method	Test Time	Train Time	IOU
(Caicedo and Lazebnik 2015)	12 s	-	0.45
(Yoo et al. 2015)	28 s	-	0.23
(Bueno et al. 2017)	10 s	27 h	0.47
RCNN(Girshick et al. 2014)	76 s	-	0.82
Our Method	8 s	15 h	0.74

Table 1: Comparison of computation efficiency and performance of various methods on PASCAL-VOC(Everingham et al. 2010) dataset. As it can be seen, our method outperforms various reinforcement learning-based methods(Bueno et al. 2017; Caicedo and Lazebnik 2015) while having a faster training time. Even though RCNN(Girshick et al. 2014) have slightly better performance, their test time is significantly higher than our method.

we compare our method with 2 other works: first with reinforcement learning-based method (Bueno et al. 2017) and the other with sequential classification method (Yoo et al. 2015). It can be seen that when it comes to satellite images, both methods have subpar results in terms of IOU, especially the reinforcement learning-based method, as the state space is very huge. The training is very challenging with a direct Q network. AttentionNet works relatively better but is still under the desired performance. Both methods have huge computation time in training and testing as they directly deal with high-resolution images. Our method, however, deals with lower-resolution images first, then upscales

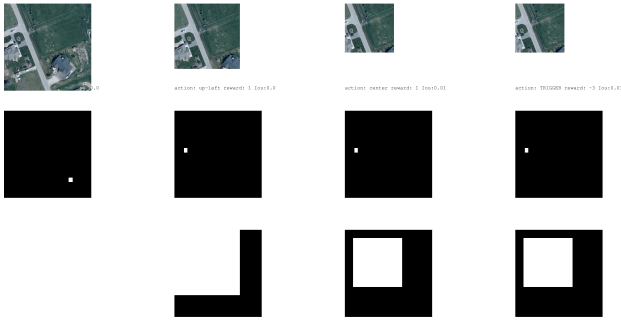


Figure 5: A bad sample run of the CDQN agent on the VEDAI dataset. The agent asks for a higher-resolution image and terminates quickly. The agent seems to fail in regions where the context is difficult to understand. For example, farm trucks are hard to perceive by an agent as it seems to focus towards roads more than farms.

to higher-resolution patches that are relatively small-sized. Our method has much better IOU scores but can be improved further with the change in termination threshold from 0.5 to higher values. Due to time constraints, we could not train efficient networks as higher thresholds required more training time and resources.

Method	Test Time	Train Time	IOU
(Bueno et al. 2017)	23 s	28 h	0.19
(Yoo et al. 2015)	47 s	29 h	0.24
Our Method	18 s	17 h	0.49

Table 2: Comparison of computation efficiency and performance of various methods on VEDAI(Razakarivony and Jurie 2016) dataset. As can be seen, our method outperforms other methods significantly both in terms of time and IOU. We note that the IOU threshold can be increased with more computational time and resources to improve the overall IOU scores.

Effect of history vector

Previous works (Bueno et al. 2017) have suggested the usage of history vectors in state space. But we realise that the states are self-sufficient to be markovian, and they do not require history information to be markovian. We, therefore, wanted to analyse the effect of the history vector in Table 3. It can be seen that the history vector is leading to sub-optimal IOU, and the agent is well off without the history vector, which is very interesting.

Effect of terminal reward

We analyse the effects of terminal reward. It might look like the more the reward, the better performance, but that is not simply the case. One has to consider the relative reward strengths and many other factors. We do a simple grid search of terminal reward strength and find that higher values do not necessarily mean better learning, as shown in Table 4

size of history	IOU
0 actions	0.51
1 action	0.49
4 actions	a.49
6 actions	0.32

Table 3: Effect of size of history vector on training efficiency over VEDAI dataset. It can be seen that the training is much better without any history vector

Terminal Reward	IOU
3	0.49
10	0.31
15	0.27

Table 4: Effect of terminal reward magnitude on training efficiency over VEDAI dataset. It can be seen that a higher reward does not necessarily imply better learning.

Results on PASCAL-VOC

We also present interesting results on the PASCAL-VOC dataset in Figure 6. Since the objects have a good size ratio compared to the images, it is much easier to train the agent to detect objects, as shown in Table 1.

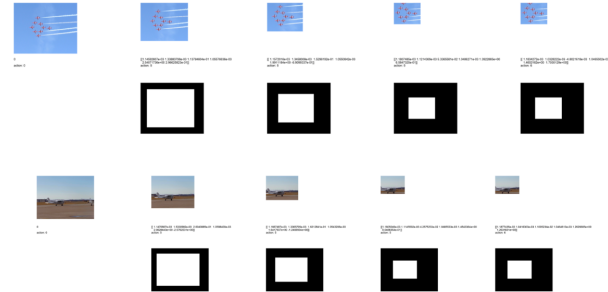


Figure 6: A few good detection samples by CDQN on the PASCAL-VOC2012 dataset. The agent sometimes seems to terminate at multi-object regions, as shown in the top example. But most cases it performs very well on natural images.

Conclusion

This work presents an efficient deep reinforcement learning-based algorithm to detect small objects in satellite images. We proposed a sequential decision-based hierarchical approach to work with multiple resolutions and contexts, making learning faster and more stable. The experiments suggest these improvements analytically and qualitatively. In the future, we would like to extend this work by improving the threshold levels and improve reward functions through thorough analysis. Our method can enable applications such as identifying and tracking objects in satellite images for surveillance, environmental monitoring, and disaster response. It can handle complex scenarios and adapt to changing conditions, making it a powerful tool for object detection in satellite images. The code and other details are available at <https://github.com/RohitGandikota/detection-2022-CDQN>

References

- Ba, J.; Kiros, R.; Cho, K.; Courville, A.; Salakhutdinov, R.; Zemel, R.; and Bengio, Y. 2015. Tell: Neural Image Caption Generation with Visual Attention Kelvin Xu. *arXiv (2015-02-10)* <https://arxiv.org/abs/1502.03044> v3.
- Ba, J.; Mnih, V.; and Kavukcuoglu, K. 2014. Multiple object recognition with visual attention. *arXiv preprint arXiv:1412.7755*.
- Bueno, M. B.; Giró-i Nieto, X.; Marqués, F.; and Torres, J. 2017. Hierarchical object detection with deep reinforcement learning. *Deep Learning for Image Processing Applications*, 31(164): 3.
- Caicedo, J. C.; and Lazebnik, S. 2015. Active object localization with deep reinforcement learning. In *Proceedings of the IEEE international conference on computer vision*, 2488–2496.
- Everingham, M.; Van Gool, L.; Williams, C. K. I.; Winn, J.; and Zisserman, A. 2010. The Pascal Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision*, 88(2): 303–338.
- Gandikota, R.; Sharma, A.; ManjuSarma, M.; Bothale, V. M.; et al. 2020. RTC-GAN: REAL-TIME CLASSIFICATION OF SATELLITE IMAGERY USING DEEP GENERATIVE ADVERSARIAL NETWORKS WITH INFUSED SPECTRAL INFORMATION. In *IGARSS 2020-2020 IEEE International Geoscience and Remote Sensing Symposium*, 6993–6996. IEEE.
- Girshick, R.; Donahue, J.; Darrell, T.; and Malik, J. 2014. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 580–587.
- Mnih, V.; Heess, N.; Graves, A.; et al. 2014. Recurrent models of visual attention. *Advances in neural information processing systems*, 27.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533.
- Razakarivony, S.; and Jurie, F. 2016. Vehicle detection in aerial imagery: A small target detection benchmark. *Journal of Visual Communication and Image Representation*, 34: 187–203.
- Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; Van Den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; et al. 2016. Mastering the game of Go with deep neural networks and tree search. *nature*, 529(7587): 484–489.
- Van de Sande, K. E.; Uijlings, J. R.; Gevers, T.; and Smeulders, A. W. 2011. Segmentation as selective search for object recognition. In *2011 international conference on computer vision*, 1879–1886. IEEE.
- Yeung, S.; Russakovsky, O.; Mori, G.; and Fei-Fei, L. 2016. End-to-end learning of action detection from frame glimpses in videos. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2678–2687.
- Yoo, D.; Park, S.; Lee, J.-Y.; Paek, A. S.; and So Kweon, I. 2015. Attentionnet: Aggregating weak directions for accurate object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, 2659–2667.
- Zitnick, C. L.; and Dollár, P. 2014. Edge boxes: Locating object proposals from edges. In *European conference on computer vision*, 391–405. Springer.

Acknowledgments

The authors would like to acknowledge Prof. Chris Amato for his immense support and suggestions during the entire design process of this work. We would also like to show our acknowledgement towards the TA staff who were constantly supporting our work and helping in understanding the basic concepts that underline this work. Finally, we thank our classmates for the insightful discussions about reinforcement learning concepts.